



Unit I: Heap Structures

Fibonacci Heaps

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

What is a Fibonacci Heap?

A **Fibonacci Heap** is a collection of min-heap-ordered trees. It provides faster amortized running times for several key operations compared to binary or binomial heaps.

Core Structure & Principles

- **Root List:** Roots of all trees are linked using a circular doubly-linked list.
- **Min Pointer:** Points to the minimum root in the root list. Gives $O(1)$ time access to the minimum element.
- **Relaxed Structure:** Trees do not have to be binomial. Consolidation of trees is deferred until the Extract-Min operation.
- **Node Pointers:** Each node contains parent, child (leftmost), left/right siblings, degree, and a mark flag.

Why 'Fibonacci'?

- **Degree Bounds:** The maximum degree of any node in a heap of size n is bounded by $O(\log n)$.
- **Golden Ratio relation:** If a node in a Fibonacci heap has degree d , the subtree rooted at that node has size at least $F_{(d+2)}$ (the $(d+2)$ -th Fibonacci number).
- **Size growth:** Since Fibonacci numbers grow exponentially ($F_k \approx \phi^k$, where $\phi \approx 1.618$), the minimum size of a tree grows exponentially, bounding the maximum degree.
- **Node Marking:** A node is marked if it has lost a single child since it was made a child of another node. Helps maintain structural balance.

Amortized Complexity Analysis

Comparison of asymptotic running times between different heap types:

Operation	Binary Heap (Worst)	Binomial Heap (Amortized)	Fibonacci Heap (Amortized)
Insert	$O(\log n)$	$O(1)$	$O(1)$
Find Min	$O(1)$	$O(\log n)$ or $O(1)$	$O(1)$
Union	$O(n)$	$O(\log n)$	$O(1)$
Extract Min	$O(\log n)$	$O(\log n)$	$O(\log n)$
Decrease Key	$O(\log n)$	$O(\log n)$	$O(1)$

Extract-Min: Consolidating Trees

Extract-Min is the most complex operation. It extracts the min node and cleans up the root list:

Step-by-Step Extract-Min Process

- 1. Remove the Minimum Node:** Take out the node pointed to by the min pointer.
- 2. Promote Children:** Add all children of the removed min node to the root list. They become root trees.
- 3. Consolidate Root List:** Reduce the number of trees in the root list. We link trees of the same degree:
 - Create an auxiliary array of size $O(\log n)$ indexed by degree.
 - Traverse the root list. If two trees have the same degree d , make the one with the larger root key a child of the other, increasing its degree to $d+1$.
 - Repeat this linking process until there is at most one tree of each degree.
- 4. Re-establish Min Pointer:** Scan the remaining consolidated roots to find the new minimum node.

Decrease-Key: Cascading Cuts

Decrease-Key reduces a node's key value. If the heap order is violated, it performs cuts:

The Cut & Cascading Cut Algorithm

If node x 's new key is less than its parent y 's key, heap order is violated:

- 1. Cut Node x :** Sever the link between x and y . Add x to the root list. Clear its mark flag.
- 2. Cascading Cut on Parent y :** Check parent y :
 - If y is a root, do nothing.
 - If y is unmarked, mark it (indicates it has lost a child).
 - If y is already marked, cut y from its parent, add y to the root list (unmarked), and recursively apply cascading cut to y 's parent.

Intuition & Purpose

- **$O(1)$ Amortized Time:** By cutting immediately, we avoid traversing the tree to bubble up. The amortized cost of cascading cuts balances out over time.
- **Balanced Subtrees:** Node marking guarantees that no node loses more than one child while remaining a child itself. This preserves the exponential size property of trees relative to their degree.
- **Fibonacci Relation:** This specific rule ensures that a node of degree d has a subtree of size at least ϕ^d , keeping the maximum degree bounded by $O(\log n)$.

Applications in Computer Science

Fibonacci heaps are primarily used in graph algorithms where Decrease-Key is frequent:

Dijkstra's Shortest Path

Dijkstra's algorithm finds the shortest path from a source node to all others. Using a Fibonacci heap, the complexity is reduced from $O(E \log V)$ to $O(E + V \log V)$. This is highly efficient for dense graphs with many edges.

Prim's Minimum Spanning Tree

Prim's algorithm finds the MST of a weighted graph. Using a Fibonacci heap to store vertices ordered by connection weight improves the runtime to $O(E + V \log V)$, matching Dijkstra's performance.

Network Flow Algorithms

Several advanced network flow and matching algorithms rely on min-priority queues. Using Fibonacci heaps speeds up the bottleneck operations of finding augmenting paths.

Theoretical Algorithmics

Fibonacci heaps are used as a building block in many advanced data structures, such as directed minimum spanning trees, and link-cut trees, where theoretical efficiency is critical.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management